

inference-engine

A from-scratch LLM serving engine — Deep Dive

Working explainer document (not published website content)

What this document is

An exhaustive, beginner-safe walkthrough of the **inference-engine** project: a GPT-2 inference and serving engine built from scratch in Python and PyTorch. Every claim below is drawn directly from the project's source code, README, and architecture notes — nothing is invented. Every technical term is defined the first time it is used. This is a working document for the owner's own understanding and interview preparation; it is not part of the published portfolio website.

Contents

1	What the project is, and why it exists	2
1.1	The feature set, at a glance	3
2	Package layout	3
3	The four-layer architecture	3
3.1	Why this shape	3
4	Request lifecycle	5
4.1	Step by step	5
5	Sequences: the unit of work	7
6	The paged KV cache	7
6.1	The problem with a naive cache	7
6.2	Paging: the fix	8
6.3	How addressing works	8
6.4	BlockManager: metadata, not tensors	8
6.5	Copy-on-write (COW)	9
6.6	Prefix caching	9
6.7	The rewritten-block bug this design had to solve	10
7	Continuous batching and the scheduler	10
7.1	Static batching's problem	10
7.2	Continuous batching's fix	10
7.3	Scheduler mechanics	10

7.4	Preemption	11
8	The model: GPT-2 forward pass and paged attention	12
8.1	Loading pretrained weights	12
8.2	ForwardContext and the ModelRunner	12
9	Sampling: turning logits into a token	13
10	Speculative decoding	13
10.1	The mechanism	14
10.2	Rollback	14
10.3	Why the draft cache skips prefix caching	14
10.4	Measured result: honest, not flattering	15
11	The HTTP server	15
12	Correctness verification: the logit-parity test	16
13	The test suite	16
14	Challenges encountered (from the README)	16
15	Benchmarks	17
15.1	Throughput and time-to-first-token vs. concurrency	17
15.2	Batched engine vs. sequential Hugging Face <code>generate()</code>	17
15.3	Speculative decoding on vs. off	18
16	What was learned, and what would be done differently	18
17	Glossary	19

1 What the project is, and why it exists

`inference-engine` is a Python + PyTorch implementation of the core mechanisms that power modern **LLM serving engines** (systems like vLLM that run a large language model and answer many users' requests at once, efficiently). It serves GPT-2 family models (`gpt2`, `distilgpt2`) over an OpenAI-compatible HTTP API, running entirely on CPU.

Term: LLM

A **Large Language Model** is a neural network trained to predict the next token (a word-piece or sub-word unit) given a sequence of previous tokens. Text is generated by repeatedly predicting one token, appending it, and repeating.

Term: Inference vs. training

Training adjusts a model's weights from data. **Inference** is using an already-trained, frozen model to produce outputs. This project is entirely about inference: how to run a trained GPT-2 model quickly and serve many concurrent users.

Hugging Face's `transformers` library is used *only* to download the pretrained GPT-2 weights from

the internet. The forward pass (the arithmetic that turns input tokens into next-token predictions), the attention mechanism, the KV cache, and the request scheduler are all implemented in this repository, not borrowed from a library. The README states the motivation directly: reading a paper about how vLLM works is one thing; rebuilding the mechanisms small enough to fit in one head — and verifying them against a reference implementation — is another.

1.1 The feature set, at a glance

- An independent GPT-2 forward implementation, numerically verified against Hugging Face **transformers** (maximum absolute logit difference $< 10^{-3}$; measured $\approx 3 \times 10^{-5}$ on **distilgpt2** and $\approx 2 \times 10^{-4}$ on **gpt2** in float32).
- A **paged KV cache**: fixed-size memory blocks, reference-counted free lists, per-sequence block tables, and copy-on-write when a shared block is written.
- **Prefix caching**: identical prompt prefixes reuse already-computed cache blocks instead of recomputing them.
- **Continuous batching**: requests join and leave the running batch at every token boundary, with preemption when memory runs short.
- **Speculative decoding**: a small “draft” model proposes several tokens at once, verified by the larger “target” model in a single forward pass.
- Standard sampling controls: greedy decoding, temperature, top-k, top-p (nucleus) sampling, repetition penalty, seeded determinism, and $n > 1$ parallel completions per prompt.
- An HTTP server (FastAPI) exposing an OpenAI-compatible subset: `/v1/completions` with Server-Sent Events (SSE) streaming, `/v1/models`, `/healthz`, `/metrics`.

2 Package layout

3 The four-layer architecture

Term: Layered architecture

Splitting a system into layers means each layer only talks to the one directly below it, and only through a narrow, well-defined interface. Here: HTTP concerns never leak into scheduling logic, and scheduling logic never leaks into tensor math.

3.1 Why this shape

- **server/app.py** only knows about HTTP: parsing JSON requests, validating them, and turning engine output into either a single JSON response or a stream of Server-Sent Events.
- **AsyncEngine** exists because Python’s asyncio event loop (the thing that lets one process juggle many network connections without threads) must never be blocked by CPU-heavy work. Running a transformer forward pass takes real wall-clock time, so it is pushed onto a worker thread with `asyncio.to_thread`, keeping the event loop free to accept new connections and stream tokens to existing ones.

```

inference-engine/
├── README.md (usage, design notes, benchmarks)
├── docs/ARCHITECTURE.md (layered diagram, tradeoffs)
├── engine/ (the core — no HTTP, no I/O)
│   ├── config.py EngineConfig, SamplingParams
│   ├── sequence.py Sequence, status/finish enums
│   ├── block_manager.py paging, refcounts, prefix cache, COW
│   ├── cache.py physical KV tensors
│   ├── model.py GPT2 forward, paged attention, ModelRunner
│   ├── sampling.py temperature/top-k/top-p/rep. penalty
│   ├── scheduler.py admission, batching, preemption
│   ├── speculative.py draft/verify accept-reject sampling
│   ├── llm_engine.py synchronous engine core, step()
│   └── async_engine.py background loop, per-seq queues
├── server/ (HTTP surface)
│   ├── app.py FastAPI routes, SSE fan-out
│   └── __main__.py CLI / env-var entrypoint
├── tests/ 55 collected tests across 8 files
└── benchmarks/bench.py throughput/TTFT/speculative harness

```

Figure 1: Directory layout. `engine/` is a plain Python library with no network code; `server/` is the thin HTTP shell around it. This separation means the whole engine can be tested and driven directly in Python (as the test suite does) without ever starting a web server.

- **LLMEngine** is the synchronous heart of the system: it has no knowledge of HTTP or asyncio at all. It can be driven directly from a test or a benchmark script by repeatedly calling `step()`.
- **Scheduler** and **BlockManager** cooperate: the scheduler decides *which* sequences run next, the block manager tracks *where* their data lives in cache memory and whether there is room.
- **ModelRunner** + **GPT2** is the only place tensors actually get multiplied; everything above it deals in token ids, block tables, and Python objects.

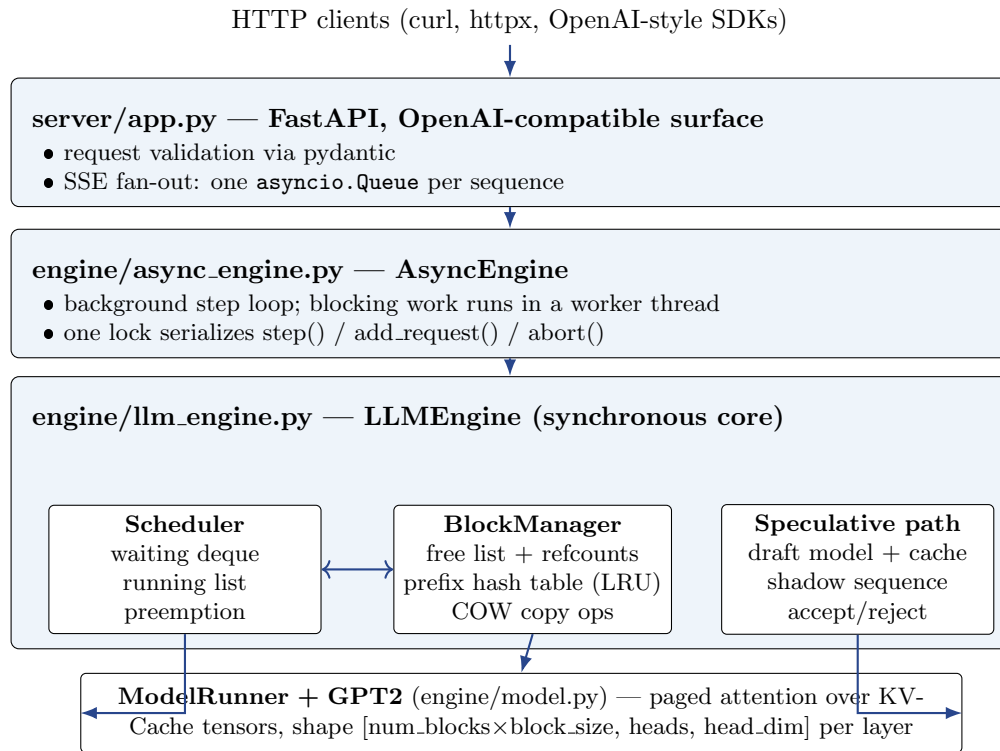


Figure 2: The four layers requests pass through, top to bottom, matching `docs/ARCHITECTURE.md`. Tokens flow back up the same path as they are produced.

4 Request lifecycle

Term: Token

GPT-2 does not operate on raw characters or whole words. Text is split into **tokens** — sub-word units from a fixed vocabulary — by a **tokenizer**. “Token” is used throughout this document to mean one such unit; a token id is its integer index into the vocabulary.

4.1 Step by step

1. A client POSTs to `/v1/completions` with a JSON body (`server/app.py`’s `CompletionRequest` pydantic model: `prompt`, `max_tokens`, `temperature`, `top_p`, `top_k`, `repetition_penalty`, `n`, `stream`, `stop`, `seed`).

Term: pydantic

A Python library that validates incoming data against declared types and constraints (e.g. `ge=1` means “must be ≥ 1 ”) and rejects malformed requests automatically.

2. The route converts the request into a `SamplingParams` object and calls `engine.submit(...)` on the shared `AsyncEngine`.
3. `AsyncEngine.submit` runs `LLMEngine.add_request` on a worker thread (under its lock), which tokenizes the prompt (if it was given as a string), validates length limits, creates one `Sequence` object per requested completion (`n`), and enqueues the first (“parent”) sequence on the scheduler.

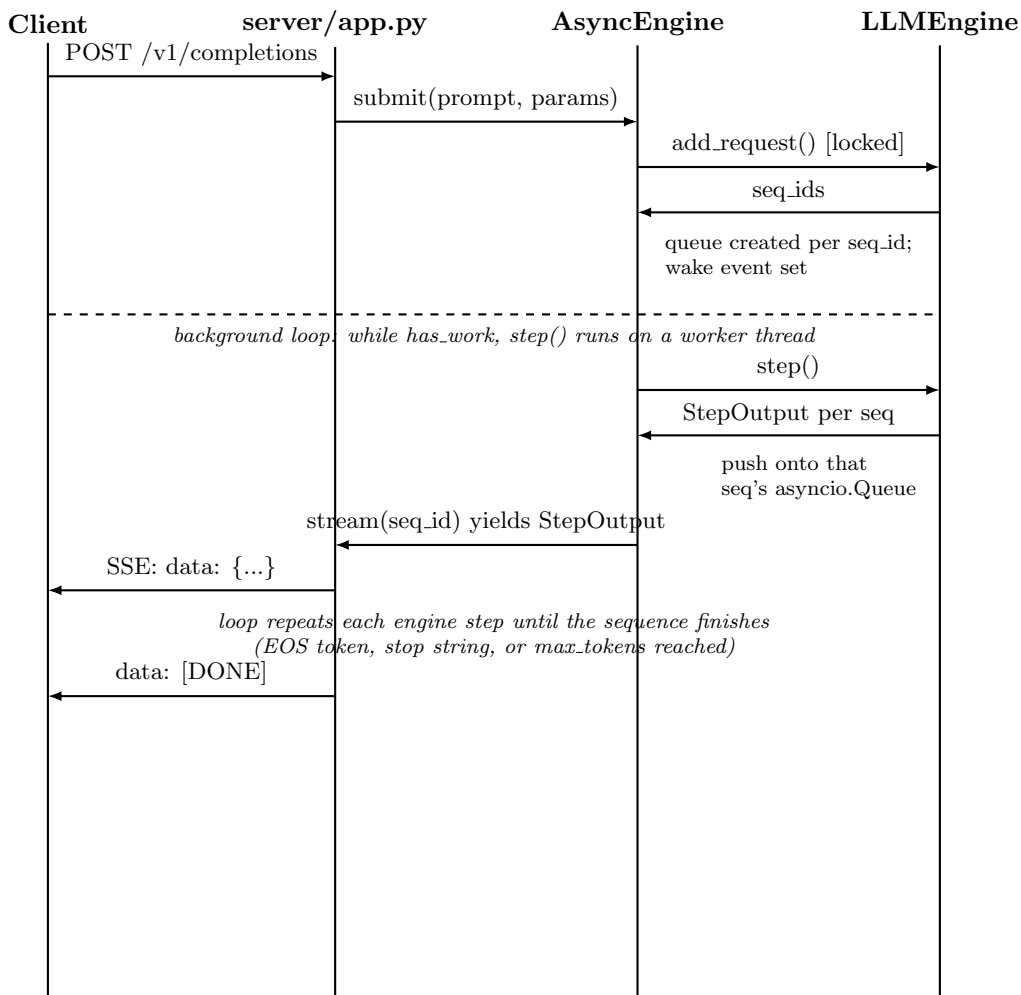


Figure 3: One completion request’s path through the system, from HTTP call to the final SSE event. Non-streaming requests follow the same path but the server simply waits for all deltas before returning one JSON body.

Extra siblings ($n > 1$) are held as *pending forks* to be split off the parent cheaply after its prompt has been processed once (see §6.5).

4. One `asyncio.Queue` is created per returned `seq_id`, and a wake event tells the background loop there is work.
5. The background loop (`AsyncEngine._run_loop`) repeatedly calls `LLMEngine.step()` on a worker thread whenever there is work, and pushes each sequence’s `StepOutput` onto its queue.
6. The HTTP handler (`_sse_stream` for streaming, or a simple loop for non-streaming) drains each sequence’s queue and turns `StepOutputs` into either SSE `data:` frames or, at the end, one JSON completion object.
7. When a sequence’s `StepOutput.finished` is true (end-of-sequence token, a stop string matched, or `max_tokens` reached) the stream for that sequence ends; once all requested completions are done the SSE stream sends a final `data: [DONE]` and closes.
8. If the client disconnects mid-stream, `server/app.py` notices via `Request.is_disconnected()`,

cancels its background tasks, and calls `engine.abort(seq.id)` for every sequence in the request so the engine reclaims its cache memory immediately instead of continuing to generate tokens nobody will read.

5 Sequences: the unit of work

Term: Sequence

A **Sequence** (`engine/sequence.py`) is one generation stream: a list of token ids (prompt tokens followed by generated tokens), its sampling parameters, its status, and its **block table** (§6).

Every sequence tracks `num_computed`: how many of its tokens already have their attention Key/Value entries written into the cache. The engine’s core invariant between steps is `num_computed == len(token_ids) - 1`: the newest token has been appended to the list but not yet fed through the model. A sequence moves through three statuses: **WAITING** (queued, not yet admitted), **RUNNING** (in the active batch), **FINISHED** (done, either because it hit `max_tokens` — `FinishReason.LENGTH` — or produced an end-of-sequence token or matched a stop string — `FinishReason.STOP`).

`SamplingParams` (`engine/config.py`) holds every per-request knob: `max_tokens`, `temperature` (0.0 means greedy — always pick the single most likely next token), `top_k`, `top_p`, `repetition_penalty`, an optional `seed` for reproducibility, a list of `stop` strings, and `n` (how many independent completions to generate for one prompt).

6 The paged KV cache

This is the project’s central data-structure idea, borrowed deliberately from operating-systems memory management (the same idea vLLM’s `PagedAttention` is named after).

Term: Attention, KV cache

A transformer’s **attention** mechanism lets each new token look back at every previous token’s **Key** and **Value** vectors to decide what to attend to. Recomputing every previous token’s Key/Value on every single new token would be wasteful, so they are cached the first time they are computed — this cache is the **KV cache**.

6.1 The problem with a naive cache

A naive implementation reserves one contiguous block of memory sized `max_len` for every sequence, whether or not it ever generates that many tokens. Two costs follow: memory is claimed for tokens that may never be generated, and a request with a large `max_tokens` monopolizes that memory even while other, shorter requests finish and could use it.

6.2 Paging: the fix

Term: Page/Block

Operating systems solve exactly this fragmentation problem by carving physical memory into fixed-size **pages** and giving each process a **page table** that maps its logical addresses to physical page numbers, wherever they happen to be. This project’s **block** is the same idea applied to KV cache memory: a fixed-size chunk (16 tokens by default, `block_size` in `EngineConfig`) that can hold any sequence’s tokens, addressed through a per-sequence **block table** (a Python list of block ids) instead of one big contiguous region.

Physical KV block pool (`num_blocks` fixed-size slots)

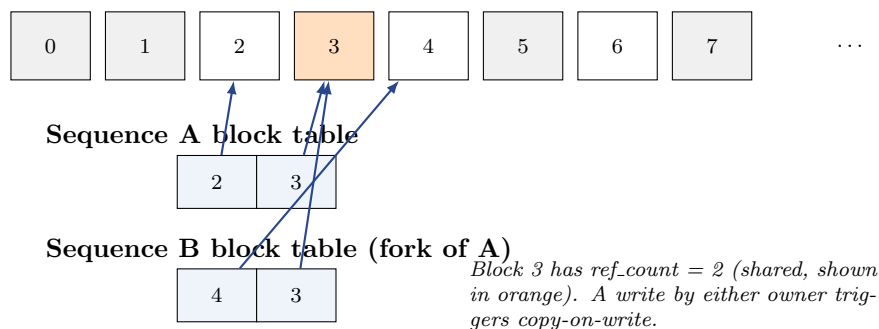


Figure 4: Two sequences with independent block tables. Block 3 is shared (reference count 2) because sequence B is a fork of sequence A — e.g. an $n > 1$ sibling completion, or a prefix-cache hit. Only a write into a shared block forces a physical copy.

6.3 How addressing works

The physical KV tensors (`engine/cache.py`, class `KVCache`) are shaped `[num_blocks * block_size, num_heads, head_dim]` per transformer layer, so a single flat integer — the **slot** — addresses one token’s Key or Value row. Token position p within a sequence maps to slot

$$\text{block_table}[p // \text{block_size}] \times \text{block_size} + (p \bmod \text{block_size})$$

(`BlockManager.slot_mapping`). `KVCache.write` uses `index_copy_` to scatter new Key/Value rows into arbitrary slots in one call; `KVCache.gather` reads rows back out the same way; and `KVCache.copy_blocks` performs copy-on-write by literally copying one block-sized tensor slice onto another, across every layer, when a block is duplicated.

6.4 BlockManager: metadata, not tensors

`engine/block_manager.py`’s `BlockManager` owns *only* bookkeeping — which blocks are free, which are shared, which hold which prefix’s data — and never touches a tensor directly. It hands the `ModelRunner` lists of `(src_block, dst_block)` copy instructions and lets the runner apply them to the actual cache. This separation is called out explicitly in the README as a design choice: it kept both halves (allocation logic and tensor math) independently testable, and the project’s test suite exercises the `BlockManager` entirely without ever loading a model.

Each `Block` has a `block_id`, a `ref_count` (how many sequences currently point at it), and an optional `content_hash` (set once the block is completely full of tokens). Free blocks live in a deque

(`free_ids`); blocks whose reference count has dropped to zero *but which are still hash-registered* move to an `evictable` ordered dict acting as an LRU (least-recently-used) pool instead of being reused immediately — this is what makes prefix caching possible (§6.6). `num_free_blocks` counts both pools together, since an evictable block can always be reclaimed if nothing needs its cached content.

6.5 Copy-on-write (COW)

Term: Copy-on-write

A technique (named for the identical mechanism in Unix’s `fork(2)` system call, which duplicates a process’s memory) where two owners share the same physical data until one of them writes to it; only then is a private copy made. It turns an expensive eager copy into a cheap reference-count bump that is upgraded to a real copy only when necessary.

Two situations create shared blocks here: an $n > 1$ sibling forking its parent’s already-computed prompt blocks (`BlockManager.fork`), and a prefix-cache hit reusing another request’s blocks (§6.6). Before writing new tokens into a sequence’s tail blocks, `append_slots` checks each block’s `ref_count`: if it is greater than 1, a fresh block is popped from the free pool, the physical contents are scheduled for copying (returned as a `CopyOp`), the sequence’s block table is repointed at the new block, and the old block’s reference count is decremented. Only the sequence that triggers the write pays the copy cost; every other sharer is untouched.

6.6 Prefix caching

Term: Content hash

A fixed-size fingerprint (here, SHA-256, a cryptographic hash function) computed from data such that two different inputs are, for all practical purposes, guaranteed to produce different fingerprints. Comparing hashes is a fast way to check “is this the same content?” without comparing the raw data itself.

Once a block is completely filled with tokens, its content is hashed *including* the hash of every block before it in that sequence (`_hash_block` chains the previous block’s hash into the next), so identical token windows that arrive with different preceding context never collide, but identical *prefixes* do. That hash becomes the key in `hash_to_block`, a dictionary from content hash to the block id holding it.

When a new request arrives, `_match_prefix` walks its prompt’s block hashes from the start and stops at the first hash that is not already registered; the matched run of blocks is reused (their reference counts are bumped) instead of recomputed. The match is deliberately capped at `len(token_ids) - 1` tokens, because the model must still compute at least one position to produce a next-token prediction — matching literally every token would leave nothing left to run.

Freed hash-registered blocks are not returned straight to the free list; they go to the `evictable` LRU pool so a later request with the same prompt prefix can still find them. Only when the free list is empty does `_pop_free_block` evict the least-recently-used entry from that pool (incrementing the `evictions` counter exposed at `/metrics`).

6.7 The rewritten-block bug this design had to solve

Both a stop-string truncation and a rejected speculative-decoding draft need to *rewrite* tokens that a previous step had already hashed into the prefix cache. If a block's registered hash were left pointing at content that has since changed, a later, unrelated request could match that stale hash and read back the wrong tokens. The fix, called out in the README's Challenges section, is to deregister a block's hash the moment before it is rewritten (`BlockManager.append_slots` and `truncate` both call `_deregister`); the test `test_truncate_deregisters_rewritten_block` pins this behavior so a regression would fail the suite.

7 Continuous batching and the scheduler

Term: Batching

Running several inputs through a model *together*, as one matrix operation, is far more efficient on modern hardware than running them one at a time, because the fixed overhead of a matrix multiply is amortized across all of them.

7.1 Static batching's problem

The traditional approach groups a fixed set of requests, pads their sequences to equal length (so they form a rectangular tensor), and holds that entire batch until its *slowest* member finishes generating — meanwhile the hardware spends cycles computing over padding positions that carry no useful work for requests that already finished.

7.2 Continuous batching's fix

Term: Continuous batching

Instead of a fixed group, the active batch is recomputed at *every token boundary*: whichever sequences are alive right now form the batch. A new request's prompt is processed (its **prefill**) and it joins the decoding batch at the very next step; a sequence that just finished leaves immediately and its memory is released for others.

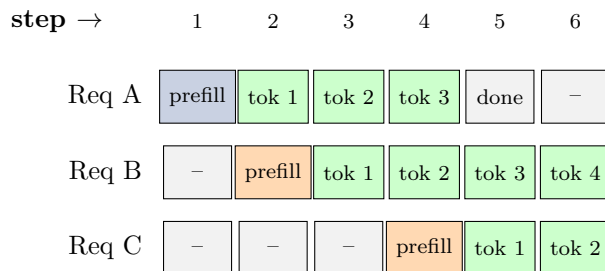
Term: Prefill vs. decode

Prefill is the one-time forward pass over an entire prompt (which may be many tokens long) to produce its first generated token and populate the KV cache for every prompt position. **Decode** is the repeated, one-token-at-a-time forward pass that follows, where each step produces exactly one new token per sequence.

7.3 Scheduler mechanics

`engine/scheduler.py`'s `Scheduler.schedule()` runs once per engine step and returns a `SchedulerOutput` (which sequences to prefill, which to decode, which were preempted):

1. **Admission.** While there is a waiting request and the running batch has room (`max_batch_size`), and the block manager confirms enough cache space for that request's prompt (`can_allocate`, which accounts for prefix-cache hits), it is popped off the FIFO waiting queue, allocated blocks, marked `RUNNING`, and added to this step's prefill list.



Blue/orange = prefill (variable-length, run alone). Green = decode (all live requests batched together in one forward). Req A finishes and frees its blocks at step 4; Req C joins at step 3 without waiting for A or B.

Figure 5: Continuous batching timeline. The decode batch’s membership changes every step; a new request’s prefill (steps 1 and 3 here) runs by itself before joining the shared decode batch.

2. **Capacity check for decode.** For every sequence that is running but was not just admitted this step, the scheduler sums (not checks individually) the worst-case number of new blocks each would need to take one more step, including potential copy-on-write copies (`BlockManager.blocks_needed`). If that summed demand exceeds the free blocks available, the sequence with the newest arrival time is **preempted**.

Why cumulative, not per-sequence

The README’s Challenges section documents a real bug the project’s own tests caught: checking each decode sequence’s capacity independently can pass every individual check while the batch *collectively* does not fit, crashing mid-step when the free list empties out. The fix sums demand across the whole batch before deciding whether anyone must be preempted; `tests/test_scheduler.py` has a dedicated “churn” test pinning this exact scenario.

7.4 Preemption

`Scheduler._preempt` frees the victim’s blocks entirely, resets its `num_computed` to 0, marks it `WAITING` again, and pushes it back onto the *front* of the waiting queue (so it is recomputed before any newer arrival). This is a **recompute** strategy rather than a **swap** strategy.

Term: Recompute vs. swap preemption

When a running request must be evicted to free memory for others, one option is to **swap** its cache to a secondary, slower memory tier and reload it later; the other is to simply **recompute** it from scratch when it re-enters. This project always recomputes, because on a CPU there is no separate “device” memory tier to swap to — device and host memory are the same, so swapping would just be extra bookkeeping around a no-op. Recomputation is also made cheap here by prefix caching: a preempted sequence’s prompt tokens are very likely to still be sitting in the evictable LRU pool.

8 The model: GPT-2 forward pass and paged attention

Term: Transformer

The neural network architecture behind GPT-2 (and virtually all modern LLMs): a stack of identical blocks, each combining an **attention** sub-layer (lets each position gather information from other positions) and a **feed-forward / MLP** sub-layer (transforms each position independently), with residual connections and layer normalization around each.

`engine/model.py` defines a plain `torch.nn.Module` implementation of GPT-2 built from four pieces:

- **Embeddings:** `wte` (token embedding: token id $\rightarrow$ vector) and `wpe` (positional embedding: sequence position $\rightarrow$ vector), summed to form each token’s initial representation.
- **Attention** (`c_attn` projects to Query/Key/Value, `c_proj` projects the attended output back down). In parity mode (no KV cache context) it calls PyTorch’s built-in `scaled_dot_product_attention` with `is_causal=True` (each position may only attend to itself and earlier positions — this is what makes GPT-2 an autoregressive, left-to-right model). In serving mode, it writes the new tokens’ K/V into the paged KVCache at `slot_mapping`, gathers back the full set of K/V rows the query needs at `gather_slots`, and builds an additive attention mask from the current position so future positions are still excluded even though the underlying tensor layout is no longer a simple contiguous sequence.
- **MLP:** a two-layer feed-forward network with a GELU activation (a smooth nonlinearity), expanding to $4\times$ the embedding width and back.
- **TransformerBlock:** LayerNorm $\rightarrow$ Attention $\rightarrow$ residual add, then LayerNorm $\rightarrow$ MLP $\rightarrow$ residual add. A **residual connection** adds a sub-layer’s input back to its output, which in deep networks stabilizes training and gradient flow — here it matters only because the pretrained weights expect it.

The final `GPT2.forward` sums the two embeddings, runs every block in sequence, applies a final LayerNorm, and computes logits (raw, unnormalized scores over the vocabulary) by multiplying by the *same* embedding matrix used at the input (**weight tying:** `x @ self.wte.weight.t()`), which is standard GPT-2 practice and saves an entire vocabulary-sized weight matrix.

8.1 Loading pretrained weights

`GPT2.from_hf` downloads the real pretrained weights through Hugging Face’s `AutoModelForCausalLM`, reads off the model’s hyperparameters (vocabulary size, embedding width, number of layers/heads) into this project’s own `ModelConfig`, and copies each weight tensor into this project’s own module. One detail matters: Hugging Face’s GPT-2 implementation stores its linear-layer weights in `Conv1D` layout, which is transposed relative to what `torch.nn.Linear` expects, so every attention/MLP weight is explicitly transposed (`.t().contiguous()`) during the copy. Getting this transpose wrong is a classic source of a model that “runs” but produces garbage — which is exactly why the project has a dedicated logit-parity test (§12).

8.2 ForwardContext and the ModelRunner

`ForwardContext` is a small bundle of paged-attention metadata (the physical cache, where to write new tokens, where to gather existing ones, and the attention mask) shared by every layer during one forward call, so each **Attention** module does not need to recompute this bookkeeping per layer.

`ModelRunner.execute` is the single place a batched forward actually runs: it requires every sequence in the batch to contribute the *same* number of new tokens T (1 for ordinary decode; $k+1$ for verifying k speculative drafts in one pass; an arbitrary equal T for same-length prefills), builds the input token ids, position ids, the flat slots to write new K/V into, the flat slots to gather existing K/V from (padded to the longest sequence in the batch, `Lmax`), and an additive causal mask that accounts for each sequence’s own already-computed offset. It marks every sequence’s `num_computed` as caught up to its current length once the forward returns.

9 Sampling: turning logits into a token

Term: Logits, softmax, temperature

Logits are the model’s raw output scores, one per vocabulary token, before they are turned into a probability distribution. **Softmax** converts logits into probabilities that sum to 1. **Temperature** divides logits before softmax: lower temperature sharpens the distribution toward the most likely tokens (temperature 0 is the limiting case, “always pick the arg-max”, i.e. **greedy decoding**); higher temperature flattens it toward more randomness.

`engine/sampling.py` implements, in order:

1. **Repetition penalty** (`adjust_logits`): logits for tokens already seen in the sequence are divided (if positive) or multiplied (if negative) by the penalty factor, discouraging the model from repeating itself. A penalty of 1.0 disables this.
2. **Temperature scaling**, skipped entirely for greedy decoding.
3. **Top-k filtering** (`filter_logits`): only the k highest-scoring tokens remain eligible; all others are set to $-\infty$ so softmax assigns them zero probability.
4. **Top-p / nucleus filtering**: tokens are sorted by probability and the smallest leading set whose cumulative probability reaches `top_p` is kept (the token that crosses the threshold is included), the rest are excluded.
5. **Sampling**: for greedy decoding, `argmax` is taken directly (no randomness involved); otherwise a categorical sample is drawn from the filtered softmax distribution via `torch.multinomial`, using a per-sequence `torch.Generator` when a `seed` was supplied, which is what makes seeded runs reproducible.

`probs_for` returns the *full* distribution (a one-hot vector at the arg-max for greedy, or the filtered softmax otherwise) rather than a single sampled token; this is the function speculative decoding needs, because it must compare the draft model’s distribution to the target model’s distribution position-by-position (§10), not just their sampled tokens.

10 Speculative decoding

Term: Speculative decoding

A technique where a small, cheap “draft” model proposes several tokens in a row, and the large “target” model **verifies all of them in a single forward pass** instead of generating them one at a time itself. When the draft guesses correctly, the target model gets several tokens for

the price of one forward pass; when it guesses wrong, only the tokens after the first mistake are discarded.

10.1 The mechanism

Every target sequence gets a **shadow sequence** (`LLMEngine._shadow`): a mirror `Sequence` object with its own block table in a separate `BlockManager` dedicated to the draft model’s cache. Each speculative round (`_speculative_decode`):

1. The draft model proposes k tokens autoregressively (one at a time, each depending on the last), recording both the sampled token and the full probability distribution q it was drawn from at each position.
2. All k proposed tokens are appended to the target sequence, and the *target* model verifies all $k+1$ positions (the k drafts plus one “bonus” position) in a single batched forward pass, producing target distributions p at each position.
3. **Accept/reject** (`speculative.accept_reject`, following Leviathan et al., 2023’s standard speculative sampling scheme): draft token x is accepted with probability $\min(1, p(x)/q(x))$. The first rejected position is replaced by sampling from the *residual* distribution $\text{normalize}(\max(p - q, 0))$ — the part of the target’s distribution the draft’s distribution under-weighted — and everything after that position is discarded (their KV cache entries are rolled back). If every draft token is accepted, one further “bonus” token is sampled from the target’s distribution at the final verified position, so a fully-accepted round still advances by $k + 1$ tokens.

Term: Why this preserves correctness

The mathematical property of this accept/reject scheme is that the tokens it ultimately emits are distributed *exactly as if they had been sampled from the target model alone*, token by token, regardless of what the draft proposed. The draft model only affects *how many forward passes* the target model needs, never *what distribution* the output comes from. At temperature 0 both distributions collapse to one-hot vectors and the rule degenerates to simply “accept while the draft’s greedy choice matches the target’s greedy choice.”

10.2 Rollback

When a draft is rejected partway through, both the target’s and the shadow’s KV caches contain entries for tokens that are being discarded. `BlockManager.truncate` drops the now-unneeded trailing blocks from the sequence’s block table (releasing them back to the free/evictable pool), and — critically, per the bug discussed in §6.6 — deregisters any prefix-cache hash that pointed at a block whose content is about to be overwritten by the eventual replacement token.

10.3 Why the draft cache skips prefix caching

The draft `BlockManager` is constructed with `enable_prefix_caching=False`. Because of this, a freed draft block always returns straight to the free list rather than lingering in an evictable pool, which guarantees the draft cache’s total block usage is always a subset of the target’s. That, in turn, means a single capacity check against the target’s cache is enough to cover both models — the draft can never be the one that runs out of room first.

10.4 Measured result: honest, not flattering

The project’s own benchmark measured speculative decoding with the target model `gpt2`, draft model `distilgpt2`, $k = 4$ drafted tokens, and greedy decoding, on the same CPU used for every other benchmark:

Mode	Throughput (tok/s)	Acceptance rate
Plain decoding	44.7	n/a
Speculative decoding	32.2	0.54

Speculative decoding is *correct* here (54% of drafted tokens accepted, identical greedy output to plain decoding) but roughly **0.7× the throughput** — slower, not faster. The reason, stated plainly in the README: on a CPU, the small draft model’s forward pass is not cheap relative to the target model’s, so the extra draft-and-verify passes cost more compute than the target-only forwards they save. Speculative decoding is a net win when the target model’s forward dominates the draft’s cost (a much larger target model, or a GPU where the draft’s extra latency is masked by parallelism) — which this CPU/GPT-2 setup is not. It remains in the codebase as a correctness-verified feature, explicitly *not* claimed as a CPU speedup.

11 The HTTP server

`server/app.py`’s `create_app` builds a FastAPI application with four routes:

- `GET /healthz` — a trivial liveness check.
- `GET /v1/models` — reports the one model this server instance is serving.
- `GET /metrics` — exposes the engine’s internal counters: free/total KV blocks, prefix-cache query and hit-token counts, copy-on-write copy count, eviction count, preemption count, and speculative drafted/accepted counts with the derived acceptance rate.
- `POST /v1/completions` — the OpenAI-compatible completion endpoint, either returning one JSON object (`stream=false`) or a `text/event-stream` of SSE frames (`stream=true`).

The FastAPI **lifespan** context manager creates the `AsyncEngine` (off the event loop, via `asyncio.to_thread`, since constructing it loads model weights) when the app starts, starts its background step-loop task, and stops it cleanly on shutdown.

Term: Server-Sent Events (SSE)

A simple, one-directional streaming protocol built on plain HTTP: the server keeps a response connection open and sends a sequence of `data: ...` frames as they become available, which the client reads incrementally instead of waiting for the whole response. It is the mechanism OpenAI’s own streaming completion API uses, which is why this project’s `stream=true` format matches it.

For $n > 1$ (multiple completions per prompt) under streaming, `_sse_stream` fans multiple sequences’ queues into one shared `asyncio.Queue` (tagging each item with its `index`) so the client receives interleaved events for every sibling completion over a single HTTP response, closing with `data: [DONE]` only once every sibling has finished.

`server/_main_.py` is the CLI entrypoint (`python -m server`); every flag (`--model`, `--draft-model`, `--block-size`, `--num-blocks`, `--max-batch-size`, `--max-model-len`, `--num-speculative-tokens`,

`--host`, `--port`) has a matching `IE_*` environment variable fallback (documented in `.env.example`), so the server can be fully configured without command-line flags, e.g. from a container’s environment.

12 Correctness verification: the logit-parity test

Before any of the serving machinery can be trusted, the model’s forward pass itself must be verified against a trusted reference — Hugging Face’s own `transformers` implementation of GPT-2. `tests/test_parity.py` runs both implementations on the same input, cache-free (no paging, no batching — a plain causal forward), and asserts the maximum absolute difference between their output logits is below 10^{-3} , parametrized over both `distilgpt2` and `gpt2`.

The measured results (not a threshold, an actual measurement, stated in the README): $\approx 3 \times 10^{-5}$ maximum absolute logit difference on `distilgpt2`, $\approx 2 \times 10^{-4}$ on `gpt2`, both comfortably inside the 10^{-3} bound, in float32 precision. The value of this test, as the README’s “What I learned” section puts it: once the forward pass itself is verified this precisely, every subsequent bug discovered while building the cache/scheduler/batching machinery is guaranteed to be a *systems* bug (wrong bookkeeping) rather than a “did I implement the model wrong” bug — which narrows debugging enormously.

13 The test suite

The project ships 8 test files under `tests/`, collecting **55 tests** in total (54 distinct `test_*` functions, one of which, `test_parity.py`’s parity check, is parametrized over two models and so counts as two test cases):

File	Focus
<code>test_parity.py</code>	Logit-level correctness against Hugging Face (§12)
<code>test_block_manager.py</code>	Allocation, refcounting, prefix cache, COW, eviction
<code>test_scheduler.py</code>	Admission, cumulative capacity checks, preemption (incl. the churn bug)
<code>test_engine.py</code>	End-to-end generation through <code>LLMEngine</code>
<code>test_sampling.py</code>	Temperature/top-k/top-p/repetition-penalty/seeding behavior
<code>test_speculative.py</code>	Accept/reject correctness, distributional guarantees, rollback
<code>test_api.py</code>	FastAPI routes, SSE streaming, async event-loop scoping
<code>conftest.py</code>	Shared fixtures

`pytest.ini` sets `asyncio_mode = auto`, which is what enables `test_api.py`’s async test functions to run without extra decorators; one of the Challenges the README documents by name is exactly about this file (§14).

14 Challenges encountered (from the README)

- **Prefix-cache correctness with in-place rewrites** — covered in §6.6; fixed by deregistering a block’s hash the moment before it is rewritten, pinned by `test_truncate_deregisters_rewritten_block`.
- **Cumulative vs. per-sequence capacity checks** — covered in the scheduler section; the original per-sequence check admitted batches that collectively did not fit and crashed mid-step; fixed by summing demand across the whole batch, caught by the scheduler’s churn test.

- **Async test hangs.** The server's background step loop needs the test fixture and every test to share a single asyncio event loop; the default per-test loop caused the API test suite to hang silently. Pinning `loop_scope="module"` fixed it, turning a suite that never finished into one taking 5 seconds for 10 tests.
- **UTF-8 across token boundaries.** GPT-2's byte-level BPE tokenizer can split one multi-byte UTF-8 character across two separate tokens, so decoding tokens one at a time can momentarily emit an invalid character (U+FFFD, the Unicode replacement character). The `Detokenizer` in `engine/llm_engine.py` holds back any trailing partial character (checking for a decoded string ending in U+FFFD) until enough tokens have arrived to decode it cleanly.

Term: Byte-pair encoding (BPE)

GPT-2's tokenizer builds its vocabulary by repeatedly merging the most frequent adjacent byte/character pairs in its training data. Because it operates at the byte level, some Unicode characters that take multiple bytes to encode can end up split across two token boundaries, which is exactly the source of the UTF-8 challenge above.

15 Benchmarks

All numbers below were measured on the project's own machine (Intel i7-1185G7, 8 threads, 32 GB RAM, PyTorch CPU float32, greedy decoding, 32 new tokens per request unless noted) via `benchmarks/bench.py`, with raw JSON in `benchmarks/results/`. They are stated in the README as *relative* comparisons on one CPU, not absolute serving-performance claims.

15.1 Throughput and time-to-first-token vs. concurrency

Concurrency	Throughput (tok/s)	TTFT mean (s)	TTFT p95 (s)
1	44.7	0.07	0.10
4	83.8	0.27	0.32
8	113.5	0.54	0.54
16	108.5	1.27	1.27

Term: TTFT

Time-to-first-token: the wall-clock delay between a request arriving and its *first* generated token being returned to the client — the latency a user actually perceives before anything appears.

Continuous batching turns extra concurrent load into roughly $2.5\times$ more aggregate throughput up to 8 in-flight requests (matching the configured `max_batch_size`), then flattens (and TTFT keeps climbing) as the 8-thread CPU saturates and additional requests simply queue longer — an expected, and honestly reported, tradeoff.

15.2 Batched engine vs. sequential Hugging Face `generate()`

Setup	Throughput (tok/s)
<code>transformers.generate()</code> , sequential	40.0
This engine, 1 concurrent request	44.7
This engine, 8 concurrent requests	113.5

At equal settings (8 requests, 32 tokens, greedy) the engine matches Hugging Face’s own `generate()` one request at a time, and is $\approx 2.8\times$ **faster** once it can batch all 8 requests together concurrently.

15.3 Speculative decoding on vs. off

Already covered in full in §10: plain 44.7 tok/s vs. speculative 32.2 tok/s (54% acceptance), a $\approx 0.7\times$ slowdown on this CPU setup — reported as-is, not spun positive.

On the absence of a vLLM comparison

The README explicitly notes that a comparison against a production engine like vLLM is deliberately absent here: vLLM requires a GPU, this development machine has none, and any such number would have had to be fabricated. That comparison is named as honest future work, not glossed over.

16 What was learned, and what would be done differently

Directly from the README, unmodified in substance:

- Paged attention turned out to be mostly a bookkeeping problem, not a math problem — the attention kernel itself barely changes, but the allocator, reference counting, and hashing beneath it are where the actual design effort lives.
- Continuous batching’s throughput win is real and measurable even on a CPU ($2.5\times$ from concurrency 1 to 8), and it comes entirely from not wasting compute on padding or on waiting for the slowest request.
- Speculative decoding’s payoff is conditional on the draft model being much cheaper than the target; measuring it honestly on CPU showed it going *backwards*, a more useful result than assuming a paper’s GPU numbers would transfer unchanged.
- Verifying the forward pass against a reference early (logits to $< 10^{-3}$) meant every later bug was guaranteed to be a systems bug, never a “did I get the model wrong” bug.

What the README names as honest limitations, left as-is rather than hidden:

- The attention path is not truly paged at the kernel level — it gathers each sequence’s K/V into a padded dense tensor and calls `scaled_dot_product_attention`, so memory savings are real but compute still touches padding; a fused varlen/paged kernel would fix this and was not needed at this scale.
- Prefills run one at a time; batching ragged-length prompts would cut TTFT under load but needs a flattened variable-length attention path, which the CPU-bound, compute-dominated prefill here did not justify.
- One coarse engine lock serializes all mutations — fine while the model forward dominates step time on CPU, but it would need to change for finer-grained, overlapped prefill/decode scheduling elsewhere.
- Preemption is recompute-only; a real device/host memory split would make swap-to-host worthwhile, but on this CPU-only machine device and host memory are the same, so recompute was the honest choice, not a shortcut.

- Benchmarks are single-machine, CPU-only, and show the right *relative* shapes but say nothing about absolute serving performance or GPU behavior; no production-engine comparison exists because none could be measured honestly without a GPU.

17 Glossary

Attention

Mechanism letting a token's representation incorporate information from other tokens' Key/Value vectors, weighted by relevance.

Autoregressive

Generating output one step at a time, where each new output depends on all previous outputs.

Copy-on-write (COW)

Sharing data between owners until one writes to it, at which point a private copy is made just for that writer.

Continuous batching

Recomputing which requests are in the active batch at every token step, rather than fixing the batch up front.

Content hash

A fingerprint of data (SHA-256 here) used to detect identical content cheaply.

Decode

The one-token-at-a-time generation phase after a prompt's prefill.

Greedy decoding

Always choosing the single highest-probability next token (temperature 0).

KV cache

Stored Key/Value vectors from earlier tokens' attention, reused instead of recomputed on every new token.

Logits

Raw, unnormalized model output scores over the vocabulary.

Paging / block

Fixed-size memory chunking that avoids fragmentation and enables sharing between sequences.

Preemption

Evicting a running sequence to free resources for others, to be resumed (here, recomputed) later.

Prefill

The one-time forward pass over a full prompt.

Prefix caching

Reusing already-cached KV blocks for prompts that share an identical leading sequence of tokens.

Server-Sent Events (SSE)

A one-way HTTP streaming protocol for pushing incremental server updates to a client.

Softmax

Converts a vector of scores into a probability distribution.

Speculative decoding

Using a cheap draft model to propose several tokens, verified by the target model in one pass.

Temperature

A scaling factor on logits before softmax controlling randomness of sampling.

Token / Tokenizer

The atomic unit of text a language model operates on; the component that converts text to/from token ids.

Top-k / Top-p sampling

Restricting sampling to the k most likely tokens, or to the smallest set whose cumulative probability reaches p .

Transformer

The neural network architecture (attention + feed-forward blocks) underlying GPT-2 and most modern LLMs.